Формальные грамматики

Лекция 5

Грамматика типа 0.

- К нулевому классу относятся все формальные грамматики. Элементы этого класса называются **неограниченными грамматиками** (англ. *unrestricted grammars*), поскольку на них не накладывается никаких ограничений. Они задают все языки, которые могут быть распознаны машиной Тьюринга. Эти языки также известны как **рекурсивно перечислимые** (англ. *recursively enumerable*).
- Правила можно записать в виде:
- $\alpha \to \beta \alpha \to \beta$, где α — любая непустая цепочка, содержащая хотя бы один нетерминальный символ, а β — любая цепочка символов из алфавита.
- Практического применения в силу своей сложности такие грамматики не имеют.

Пример

Правила:

$S \to aBcc$

$B \to A$

$BAA \to d$

$Ac \to B$

$A \to AAA \mid dB$

Выведем в данной грамматике строку addd:

$S \Rightarrow aBcc \Rightarrow aAcc \Rightarrow aBc \Rightarrow aAc \Rightarrow aB \Rightarrow aA \Rightarrow adB \Rightarrow adA \Rightarrow adAAA \Rightarrow addBAA \Rightarrow addd$

Тип 1. Контекстно-зависимые языки, грамматики, линейно ограниченные автоматы

Грамматику типа 1 можно определить как *неукорачивающую*, либо как *контекстно-зависимую*.

Смысл неукорачивающей грамматики заключается в том, что на правила накладывается ограничение: каждое правило вывода $\alpha \to \beta$ грамматики должно отвечать ограничению $|\alpha| \leq |\beta|$. Это означает, что выводимая с помощью правила цепочка не должна становиться короче исходной цепочки.

Правила вывода контекстно-зависимой грамматики

$$\alpha B\gamma \to \alpha\beta\gamma,$$

где B – нетерминальный символ; α, γ – произвольные цепочки из алфавита V^*; β – произвольные цепочки из алфавита V^+. Термин «контекстно-зависимые» отражает тот факт, что нетерминальный символ B можно заменить на цепочку β лишь в контексте α, γ.

В правиле $\alpha B\gamma \to \alpha\beta\gamma$ контекстно-зависимой грамматики α часто называют левым контекстом, а γ – правым контекстом. Обратим внимание, что в правиле вывода $\alpha \in V^*$ и $\gamma \in V^*$. Это означает, что один из контекстов может быть пустым. Если пустыми являются оба контекста, то грамматика, оставаясь контекстно-зависимой, относится к типу КС-грамматик.

Пример 1. КЗ грамматика

<u>Пример</u>: язык $L = \{a^n\ (b\ c)^n \mid n \geq 1\}$ может быть порожден грамматикой следующего вида:

$S \to aSBC \mid abc$

$cB \to cb$

$bC \to bc.$

Приведем вывод предложения $aabcbc$:

$$\underline{S} \Rightarrow a\underline{S}BC \Rightarrow aab\underline{cB}C \Rightarrow aabc\underline{bC} \Rightarrow aabcbc.$$

Пример 2

Контекстно-зависимый язык

$L = \{a^n b^n c^n \mid n > 0\}$ задается грамматикой с правилами:

$S \to ABC$

$B \to b$

$bD \to bb$

$EC \to DC$

$A \to a$

$CD \to ED$

$B \to ABCD$

$C \to c$

$ED \to EC.$

Грамматики типа 1

- Грамматики типа 1 и все последующие часто называют грамматиками непосредственных составляющих.
- Они обладают очень важным в лингвистическом аспекте свойством. Терминальные символы понимаются как слова некоторого языка, нетерминальные символы – как синтаксические категории (например, глагол, существительное в естественном языке; оператор, выражение в языке программирования), начальный символ – как «предложение», а выводимые терминальные цепочки – как правильные предложения данного языка.
- Тогда вывод предложения естественно интерпретируется как его синтаксическая структура, представленная в терминах непосредственно составляющих, т.е. известным в лингвистике способом.

Грамматики типа 1

- Язык, заданный КЗ-грамматикой распознается линейно-ограниченным автоматом, устроенный, как и машина Тьюринга, однако в процессе выполнения каждой команды линейно- ограниченного автомата длина цепочки, получаемой на входной ленте, не может стать длиннее исходной цепочки.
- Благодаря этому свойству задача распознавания всегда заканчивается за конечное число шагов.
- Все языки программирования являются контекстно-зависимыми языками. Однако из соображений эффективности контекстно- зависимые грамматики и линейно-ограниченные автоматы в трансляторах не используются.

Линейно ограниченный автомат

Определение:

Линейно ограниченный автомат (англ. *linear bounded automata*, *lba*) — недетерминированная одноленточная машина Тьюринга, которая никогда не покидает те ячейки, на которых размещен ее ввод.

Более формально:

Определение:

Линейно ограниченный автомат — формальная система $M = \langle Q, \Sigma, \Gamma, \delta, q_0, F \rangle$, в которой

- Q — множество состояний,
- $q_0 \in Q$ — начальное состояние,
- $F \subset Q$ — множество конечных состояний,
- Γ — алфавит допустимых символов ленты,
- $\Sigma \subset \Gamma$ — алфавит входных символов, который содержит два особых символа: $\#$ и $\$$ — левый и правый маркеры, находящиеся с самого начала на концах входной цепочки для того, чтобы предотвращать выход головки ленты за пределы участка, на котором размещается входная цепочка (считается, что маркеры могут использоваться только в этой роли: на место маркера нельзя записать какой-нибудь другой символ ленты, и никакой символ ленты не может быть заменен каким-нибудь маркером),
- δ — отображение типа $Q \times \Gamma \to 2^{Q \times \Gamma \times \{\leftarrow, \rightarrow\}}$.

Активация Windows

Связь линейно ограниченных автоматов с контекстно-зависимыми языками

Теорема:

Если L — контекстно-зависимый язык, то язык L принимается некоторым линейно ограниченным автоматом.

Тип 2. Контекстно-свободные языки, грамматики, автоматы с магазинной памятью

Правила КС-грамматики имеют вид:

$$A \to \alpha,$$

где A – нетерминальный символ, $\alpha \in V^+$. Правило подстановки применяется независимо от контекста, в котором находится нетерминальный символ A.

<u>Пример.</u> Контекстно-свободный язык $L = \{a^n\, b^n \mid n > 0\}$ определяется КС-грамматикой с правилами:

$I \to a\, I\, b$

$I \to a\, b.$

Рассмотрим вывод цепочки $a\ a\ b\ b$:

$$I \Rightarrow aIb \Rightarrow aabb.$$

Пример 3. КЗ -грамматика

Тот же язык $L = \{a^n b^n c^n \mid n > 0\}$ может быть порожден грамматикой с правилами:

$S \to aSBC$ $\quad$ $S \to aBC$ $\quad$ $CB \to BC$

$aB \to ab$ $\quad$ $bB \to bb$

$bC \to bc$ $\quad$ $cC \to cc.$

КС-грамматики и автомат с магазинной памятью

Определение. *Распознающим автоматом* для контекстно-свободных языков является автомат с магазинной памятью (или стеком) (рис. 2.2).

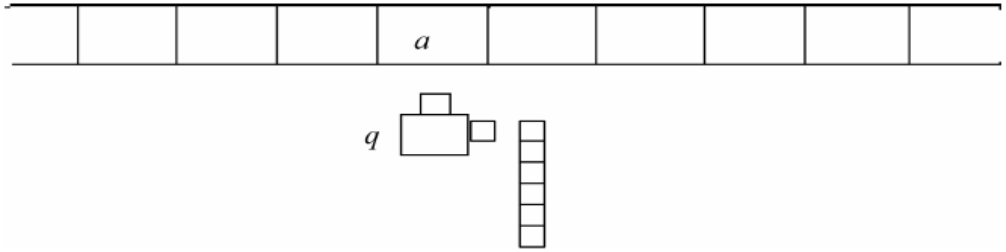


Рис. 2.2. Автомат с магазинной памятью

Конфигурация автомата определяется выражением (a, q, b). Команда автомата имеет вид $(q, a, b) \rightarrow (q_1, b_1, r)$, где a – символ входной ленты; q, q_1 – состояния автомата; b, b_1 – верхние символы магазина; r = {вправо, на месте} – продвижение по входной ленте.

Такт работы автомата следующий: если конфигурация автомата совпадает с левой частью одной из команд, то команда выполняется; выполнение команды заключается в замене верхнего символа b магазина на символ b_1, переходе из состояния q в состояние q_1, перемещении по входной ленте в соответствии со значением r.

КС-грамматики и автомат с магазинной памятью

Существенным свойством автомата с магазинной памятью является то, что направленная ко входной ленте головка предназначена только для чтения, при этом после чтения входного символа и выполнения некоторых действий с магазинной памятью в течение нескольких тактов головка чтения продвигается вперед по входной ленте. Такой принцип работы свидетельствует об эффективности автомата с магазинной памятью.

Пример

Пример. Рассмотрим задачу проверки корректности вложенности круглых скобок.

Грамматика может быть определена следующим образом:

1. $S \to (A)\ A$.

2. $A \to S$.

3. $A \to \varepsilon$.

Определим систему команд автомата следующим образом:

1. $q, (, \vdash \to q, A, R$
2. $q, (, A \to q, A, R$
3. $q,), A, \to q, \varepsilon, R$
4. $q, \$, \vdash \to$ Стоп.

Пример. Рассмотрим задачу проверки корректности вложенности круглых скобок

Определим систему команд автомата следующим образом:

1. $q, (, \vdash \rightarrow q, A, R$
2. $q, (, A \rightarrow q, A, R$
3. $q,), A, \rightarrow q, \varepsilon, R$
4. $q, \$, \vdash \rightarrow$ Стоп.

Из представленного описания видно, что поведение автомата имитирует метод распознавания с использованием счетчика, только вместо счетчика используется символ A, который записывается в стек и стирается из него, отражая разность между прочитанными открывающимися и закрывающимися скобками. Рассмотрим работу данного автомата с магазинной памятью на примере распознавания цепочки (() ()) $. Протокол работы автомата при распознавании цепочки (() ()) $ приведен в табл. 2.4.

Таблица 2.4

Номер шага	Содержимое стека	Состояние автомата	Остаток входной цепочки	Номер применяемого правила
1	$\vdash$	q	(() ()) $	1
2	$\vdash A$	q	() ()) $	2
3	$\vdash A\,A$	q	) ()) $	3
4	$\vdash A$	q	()) $	2
5	$\vdash A\,A$	q	)) $	3
6	$\vdash A$	q	) $	3
7	$\vdash$	q	$	Стоп

Тип 3. Регулярные языки, грамматики, конечные автоматы

Правила регулярной грамматики имеют вид

$$A \to aB \text{ либо } B \to a,$$

где A, B – нетерминальные символы; a – терминальный символ. Иногда первый вид правила рассматривается как $A \to Ba$, но при левостороннем чтении исходного текста это правило неудобно в использовании, поэтому такие правила рассматривать не будем.

Пример: язык $L = \{R^n \mid n > 0\}$, где $R = \{01, 10\}$ может быть задан следующей грамматикой:

$S \to 0U \mid 1Z$

$U \to 1S \mid 1$

$Z \to 0S \mid 0.$

Построим вывод цепочки 011010:

$$S \Rightarrow 0U \Rightarrow 01S \Rightarrow 011Z \Rightarrow 0110S \Rightarrow 01101Z \Rightarrow 011010.$$

Конечный автомат

С регулярной грамматикой связан конечный автомат вида (рис. 2.3).

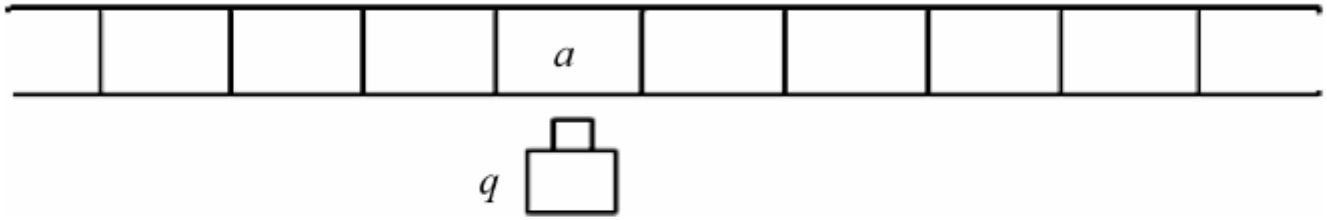


Рис. 2.3. Конечный автомат

Конечный автомат начинает работу в некотором начальном состоянии, последовательно считывая по одному символу входного слова. Считанный символ переводит автомат в новое состояние в соответствии с функцией переходов вида $(q, a) \rightarrow q_1$, где q, q_1 – состояния автомата; a – символ входной ленты.

Конечный автомат

Читая входную цепочку символов и делая переходы из состояния в состояние, автомат после прочтения последнего символа входного слова окажется в заключительном (финальном) состоянии. В этом случае говорят, что автомат допустил слово. Если автомат останавливается, не обработав входное слово до конца из-за невозможности применить функцию переходов, то такой останов свидетельствует об ошибке. В этом случае говорят, что автомат не допустил слово.

Конечные автоматы и регулярные языки

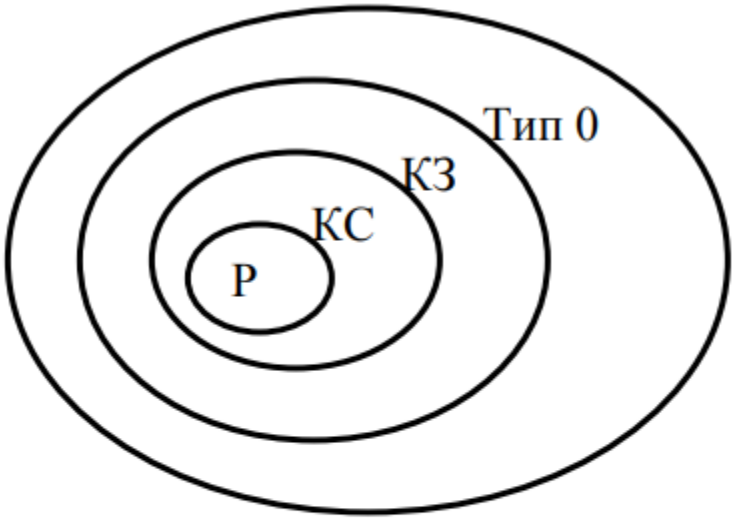
Существенным свойством конечного автомата является то, что головка предназначена только для чтения, при этом после чтения входного символа головка чтения продвигается вперед по входной ленте в соответствии с функцией переходов без всяких дополнительных действий. Число тактов автомата равно количеству символов во входном слове. Такой принцип работы свидетельствует об особой эффективности конечного автомата.

Регулярные языки активно используются для моделирования лексики формальных языков и в трансляторах.

Использование классификации формальных языков

- Формальные языки классифицируются в соответствии с типами грамматик, которыми они задаются.
- Однако один и тот же язык может быть задан разными грамматиками, относящимися к разным типам. В таком случае считается, что язык относится к наиболее простому типу.
- Язык, описанный тремя способами: грамматикой с фразовой структурой, контекстно-зависимой грамматикой и контекстно-свободной грамматикой, будет контекстно-свободным.
- Также, как и для грамматик, сложность языка определяется его типом. Наиболее сложные – языки с фразовой структурой (например, естественные языки), далее с уменьшением сложности идут контекстно-зависимые языки, контекстно-свободные языки и, наконец, самые простые – регулярные языки.
- Контекстно-свободная грамматика в ряде случаев может содержать правило вида $A \rightarrow \varepsilon$. В этом случае она называется укорачивающей контекстно-свободной грамматикой.

Соотношение между типами языков



Р – регулярная грамматика;
КС – контекстно-свободная грамматика;
КЗ – контекстно-зависимая грамматика;
Тип 0 – грамматика типа 0.

Использование классификации формальных языков

Соотношения между *типами грамматик*:

1) любая регулярная грамматика является контекстно-свободной грамматикой;

2) любая регулярная грамматика является укорачивающей контекстно-свободной грамматикой;

3) любая контекстно-свободная грамматика является контекстно-зависимой грамматикой;

4) любая контекстно-свободная грамматика является неукорачивающей грамматикой;

5) любая контекстно-зависимая грамматика является грамматикой типа 0;

6) любая неукорачивающая грамматика является грамматикой типа 0;

Соотношения между типами грамматик

7) укорачивающая контекстно-свободная грамматика, содержащая правила вида $A \to \varepsilon$, не является контекстно-зависимой и неукорачивающей грамматикой;

8) контекстно-зависимая, контекстно-свободная, регулярная грамматики являются грамматиками непосредственных составляющих.

Соотношения между типами языков

Определение: язык $L(G)$ является *языком типа* k, если его можно описать грамматикой типа k.

Соотношения между типами языков:

1) каждый регулярный язык является контекстно-свободным языком, но существуют контекстно-свободные языки, которые не являются регулярными; например контекстно-свободный язык $L = \{a^n b^n \mid n > 0\}$ не является регулярным языком;

2) каждый контекстно-свободный язык является контекстно-зависимым языком, но существуют контекстно-зависимые языки, которые не являются контекстно-свободными языками; например, контекстно-зависимый язык $L = \{a^n b^n c^n \mid n > 0\}$) не является контекстно-свободным языком;

3) каждый контекстно-зависимый язык является языком типа 0.

Замечание: следует подчеркнуть, что если язык задан грамматикой типа k, то это не значит, что не существует грамматики типа k_1 ($k_1 > k$), описывающей тот же язык. Поэтому, когда говорят о языке типа k, обычно имеют в виду минимально возможный номер k.

Соотношения языков и грамматик

Например, контекстно-свободная грамматика $G_1 = (\{0,1\}, \{S\}, P_1, S)$, контекстно-зависимая грамматика $G_2 = (\{0,1\}, \{S, A\}, P_2, S)$, укорачивающая контекстно-свободная грамматика $G_3 = (\{0,1\}, \{S, A\}, P_3, S)$, где

P_1: $S \to 0S1 \mid 01$

P_2: $S \to 0A1 \mid 01$

$0A \to 00A1 \mid 001$

P_3: $S \to 0A1$

$A \to 0A1 \mid \varepsilon$

описывают один и тот же контекстно-свободный язык $L = L(G_1) = L(G_2) = L(G_3) = \{0^n1^n \mid n > 0\}$.

Использование ФГ

Наибольшее практическое использование получили контекстно-свободная и регулярная грамматики: первая используется для построения контекстно-свободных синтаксических анализаторов языков программирования, вторая – для реализации лексического анализа языков программирования. Несмотря на то, что в задачах анализа языков программирования можно ограничиться только синтаксическим анализом на основе контекстно-свободной грамматики и автомата с магазинной памятью, введение лексического анализатора является целесообразным, так как лежащий в его основе конечный автомат эффективнее автомата с магазинной памятью.

Теория автоматов

Теория автоматов: применение

- Проектирование систем логического управления;
- Обработка текстов и построение компиляторов;
- Спецификация и верификация систем взаимодействующих процессов;
- Языки описания документов и объектно-ориентированных программ;
- Оптимизация логических программ и др.

Основные понятия и определения

- Простейший преобразователь информации отображает некоторое множество элементов информации X, поступающее на вход, в некоторое множество на выходе Y.
- Если *множества* X и Y являются конечными и дискретными, то есть преобразование осуществляется в дискретные моменты времени, то такие преобразователи информации называются конечными преобразователями.
- *Элементы множеств* X и Y в этом случае предварительно кодируют двоичными кодами и строят преобразование одного *множества* в другое.
- Результат преобразования зачастую зависит не только от того, какая *информация* в данный момент появилась на входе, но и от того, что происходило раньше, то есть от предыстории преобразования.

а)

Пример 2. "Студент“ – модель *автомата* описывает поведение студента и преподавателей

$\{S_0, S_1, \ldots, S_5, \}$ - состояния;

$\{x_1, x_2, x_3\}$ - входные сигналы: "н", "2" и "5".

$\{y_1, y_2, y_3, \ldots, y_7\}$ - выходные реакции:

- y_1 - отмечаем "н";
- y_2 - успокаивать;
- y_3 - хвалим студента;
- y_4 - поощряем;
- y_5 - надеемся;
- y_6 - предупреждаем;
- y_7 - отчисляем;

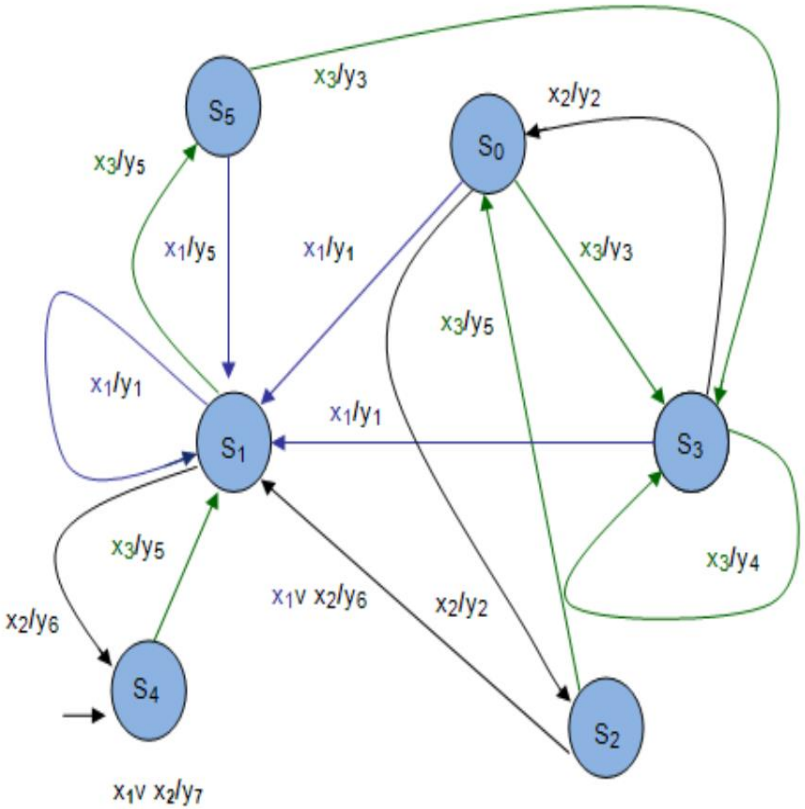
Пример 2. "Студент“ – модель *автомата* описывает поведение студента и преподавателей

$\{S_0, S_1, \ldots, S_5, \}$ - состояния;

$\{x_1, x_2, x_3\}$ - входные сигналы: "н", "2" и "5".

$\{y_1, y_2, y_3, \ldots, y_7\}$ - выходные реакции:

- y_1 - отмечаем "н";
- y_2 - успокаивать;
- y_3 - хвалим студента;
- y_4 - поощряем;
- y_5 - надеемся;
- y_6 - предупреждаем;
- y_7 - отчисляем;



Абстрактный автомат (АА)

Абстрактным автоматом называют модель, описываемую пятиместным кортежем:

$$A = (X, Y, S, f_y, f_s),$$

где первые три компонента – непустые множества:

X – множество входных сигналов АА,

Y – множество выходных сигналов АА,

S – множество состояний АА.

Два последних компонента кортежа – характеристические функции:

f_y – функция выходов;

f_s – функция переходов АА из одного состояния в другое.

Если множества X, Y, S – конечные, то такой АА называют **конечным автоматом (КА)**. Если хотя бы одно из перечисленных множеств не является конечным, то такой АА называют **бесконечным**.

Виды абстрактных автоматов

Классификация АА

I. По определенности характеристических функций.

В автоматах *полностью определенных* областью определения функций f_s и f_y является множество всех пар $(s_i, x_k) \in S \times X$, где $s_i \in S$, $x_k \in X$. В автоматах *частично определенных* либо обе характеристические функции, либо одна из них имеют областью определения строгое подмножество декартова произведения $S \times X$. Таким образом, характеристические функции подобных автоматов определены не для всех пар (s_i, x_k).

II. По однозначности функции переходов.

В *детерминированных* автоматах выполняется условие однозначности переходов: если АА находится в некотором состоянии $s_i \in S$, то под воздействием произвольного входного сигнала $x_k \in X$ автомат может перейти в одно и только одно состояние $s_j \in S$, причем ситуация $s_i = s_j$ вовсе не исключается. В автоматах *вероятностных* при воздействии одного и того же входного сигнала возможны переходы из состояния s_i в различные состояния из множества S с заданной вероятностью.

III. По устойчивости состояний:

В *устойчивых* автоматах выполняется условие устойчивости: если автомат под воздействием входного сигнала $x_k \in X$ оказался в состоянии $s_i \in S$, то выход из него и переход в иное состояние возможен только при поступлении на вход автомата другого сигнала $x_z \in X$, $x_z \neq x_k$. Если условие устойчивости не выполняется хотя бы для одного состояния $s_j \in S$, то такой автомат называют *неустойчивым*.

Недетерминированные конечные автоматы

Недетерминированный конечный автомат A=(Q, V, δ, q_0, F) представляет собой математическую модель, состоящую из

- множества *состояний* Q;
- множества входных символов V (*символов входного алфавита*);
- функции переходов δ, которая отображает пары символ-состояние на множество состояний;
- состояния q_0, известного как *стартовое* (*начальное*);
- множества состояний F, известных как *допускающие* (*конечные*);

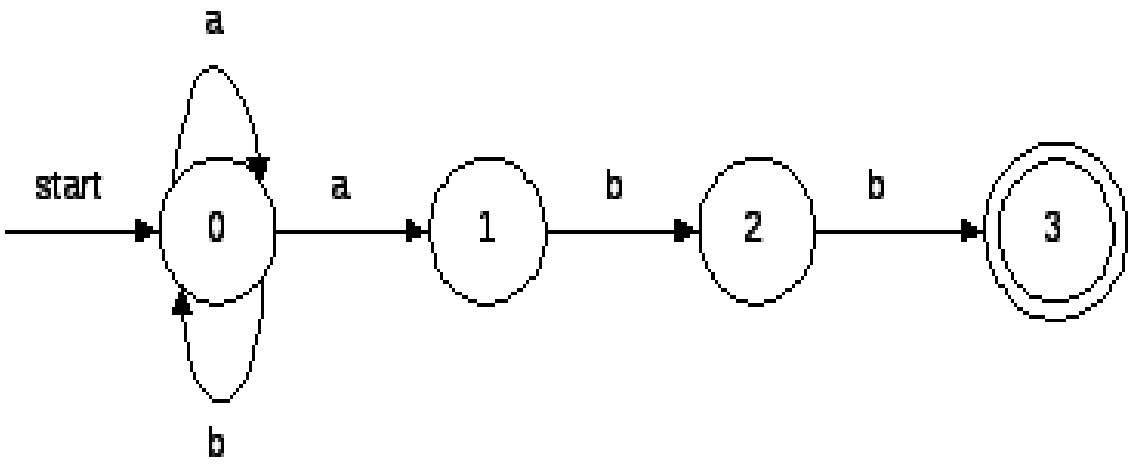
НКА может использоваться в виде помеченного ориентированного графа, так называемого *графа переходов*, узлы которого представляют собой состояния, а помеченные дуги составляют функцию переходов.

Граф переходов НКА

- Граф переходов похож на диаграмму переходов, однако один и тот же символ может помечать два и более переходов из одного состояния, а некоторые переходы могут быть помечены специальным символом ε, как обычным входным символом (ε-переходы).

Граф переходов НКА

- Граф переходов для НКА, распознающего язык *(a|b)*abb*,
- Множество состояний НКА – {0, 1, 2, 3}, а входной алфавит – {*a, b*}. Состояние 0 стартовое, а заключительное 3 представлено двойным кружком.

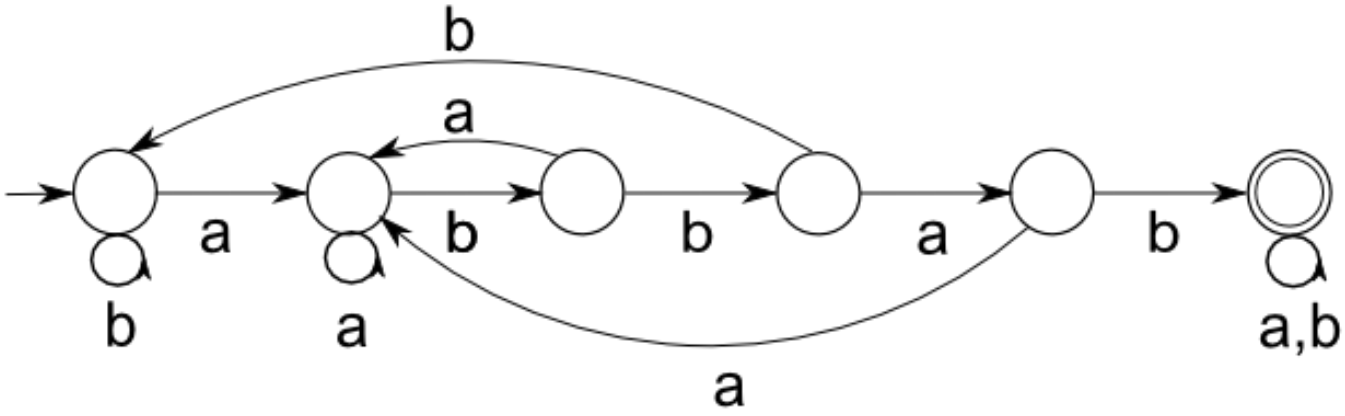


Детерминированный конечный автомат

- ***Детерминированный конечный автомат*** является специальным случаем недетерминированного конечного автомата, в котором
- отсутствуют состояния, имеющие ε —переходы;
- для каждого состояния *s* и входного символа *a* существует не более одной дуги, выходящей из *s* и помеченной как *a*.
- Детерминированный конечный автомат имеет для любого входного символа не более одного перехода из каждого состояния.

Граф переходов ДКА

Автомат для поиска образца в тексте для строки abbaabbab.



Базовые модели конечного автомата

Введем обозначения мощностей множеств, входящих в указанный выше кортеж:

$X = \{x_1, x_2, \ldots, x_p\}$, $\quad |X| = p$;

$Y = \{y_1, y_2, \ldots, y_q\}$, $\quad |Y| = q$;

$S = \{s_1, s_2, \ldots, s_n\}$, $\quad |S| = n$.

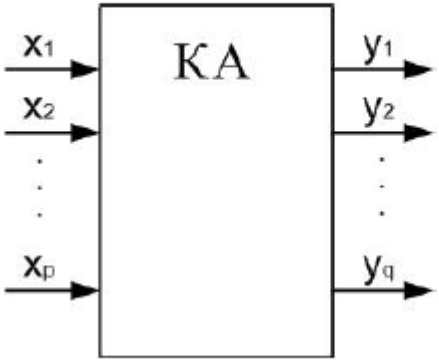
В каждый дискретный момент времени КА находится в одном и только одном состоянии S_i, воспринимает одно значение вектора X и выдает на выходе одно значение вектора Y. Принято считать, что в момент времени $t = 0$ автомат находится в начальном состоянии S_0, которое можно включить в кортеж отдельным, шестым компонентом:

$$A = (X, Y, S, f_y, f_s, S_0).$$

Автомат с выделенным начальным состоянием называют *инициальным*.

Общая схема автомата

Общую схему автомата можно представить в виде некоторого «черного ящика», осуществляющего преобразование вектора входных сигналов в вектор выходных сигналов



$$Y = f_y(X, t).$$

Характеристические функции автомата

Рассмотрим **характеристические функции** автомата.

Функция f_s реализует бинарное отношение вида $S \times X \to S$, то есть каждой паре «состояние – входной сигнал» ставит в однозначное соответствие определенное состояние из множества S. Аналогично, бинарное отношение для функции f_y имеет вид $S \times X \to Y$, то есть каждой паре «состояние – входной сигнал» ставится в соответствие конкретный выходной сигнал – элемент множества Y.

Таким образом, характеристические функции определяют, в какое состояние $s \in S$ перейдет автомат в следующий, (t+1)-й момент времени и каково будет значение выходного сигнала $y \in Y$ в текущий момент времени t:

$$\left.\begin{aligned} s(t+1) &= f_s(x(t), s(t)) \\ y(t) &= f_y(x(t), s(t)) \end{aligned}\right| \qquad (1)$$

Таким образом, характеристические функции определяют, в какое состояние $s \in S$ перейдет автомат в следующий, (t+1)-й момент времени и каково будет значение выходного сигнала $y \in Y$ в текущий момент времени t:

$$\left.\begin{aligned} s(t+1) &= f_s(x(t), s(t)) \\ y(t) &= f_y(x(t), s(t)) \end{aligned}\right| \qquad (1)$$

Из приведенных уравнений видно, что аргументами характеристических функций являются текущее значение входного сигнала и текущее состояние. Конечный автомат, заданный парой уравнений (1), называется **автоматом I рода** или, по имени автора модели, **автоматом Мили (Mealy)**.

На практике часто встречаются автоматы, выходные сигналы которых в момент времени t однозначно определяются текущим состоянием автомата и не зависят от компонентов вектора входных сигналов:

$$\left.\begin{aligned} s'(t+1) &= f_s'(x(t+1), s'(t)) \\ y(t) &= f_y'(s'(t)) \end{aligned}\right| \qquad (2)$$

Автомат, заданный парой уравнений (2), называют **автоматом II рода** или **автоматом Мура (Moore)**. Штрих введен в обозначения для отличия записи функций и состояний автомата Мура от автомата Мили. Заметим, что автомат Мили по отношению к автомату Мура «запаздывает» на один дискретный момент времени по входному сигналу.

Автомат Мили

Определение. *Конечным автоматом* (или *автоматом Мили*) называется система из пяти компонент (A, B, V, f, g), где A и B – это входной и выходной алфавиты автомата, V – множество его состояний, f – функция выходов, а g – функция переходов автомата, причем все множества A, B и V конечны.

Определение. *Функцией выходов* автомата называется функция f, которая каждой паре (a_i, v_j), где $a_i \in A$, $v_j \in V$, ставит в соответствие выходной сигнал $f(a_i, v_j) \in B$.

Функция выходов указывает, какой сигнал будет направлен на выход автомата, если он находился в состоянии v_j, и на вход ему поступил сигнал a_i.

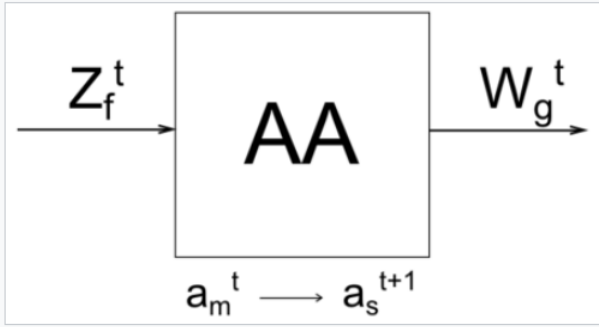
Определение. *Функцией переходов* автомата называется функция g, которая каждой паре (a_i, v_j), где $a_i \in A$, $v_j \in V$, ставит в соответствие состояние $g(a_i, v_j) \in V$.

Абстрактный автомат (англ. *Abstract Machine*) является математической моделью дискретного устройства и описывается шестикомпонентным набором $S = (A, Z, W, \delta, \lambda, a_1)$, где

- $A = \{a_1, \ldots, a_m, \ldots, a_M\}$ — множество состояний.
- $Z = \{z_1, \ldots, z_f, \ldots, z_F\}$ — множество входных сигналов.
- $W = \{w_1, \ldots, w_g, \ldots, w_G\}$ — множество выходных сигналов.
- δ — функция переходов АА, которая некоторым парам состояние — входной сигнал (a_m, z_f) ставит в соответствие состояние АА a_s, т.е. $a_s = \delta(a_m, z_f)$, $a_s \in A$.
- λ — функция выходов АА, которая некоторым парам состояние — входной сигнал (a_m, z_f) ставит в соответствие выходной сигнал АА w_g, т.е. $w_g = \lambda(a_m, z_f)$, $w_g \in W$.
- a_1 — начальное состояние. АА работает в дискретные моменты времени, и в момент времени $t = 0$ автомат всегда находится в состоянии a_1.

Выходные сигналы АА зависят от того, что поступало на его вход раньше.

В каждый момент времени АА, будучи в состоянии a_m^t, способен воспринимать одну из букв входного алфавита z_f^t. В соответствии с функцией δ, АА перейдет в состояние a_1^{t+1} с выдачей выходного сигнала, который вырабатывается в соответствии с функцией выходов λ.



Работа Абстрактного автомата

Рассмотрим функционирование автоматов Мура и Мили.

Автомат Мили

$$a(t+1) = \delta(a(t), z(t))$$
$$w(t) = \lambda(a(t), z(t))$$

Автомат Мура

$$a(t+1) = \delta(a(t), z(t))$$
$$w(t) = \lambda(a(t))$$

Способы задания автоматов

- Табличный способ задания автомата Мили
- Графический способ задания автомата Мили
- Табличный способ задания автомата Мура
- Графический способ задания автомата Мура

Табличный способ задания автомата Мили

Автомат Мили может быть задан таблицей переходов и таблицей выходов.

В таблице переходов АА Мили на пересечении столбца a_m и строки z_f записывается состояние a_s, которое есть функция δ от a_m и z_f

	a_m	
z_f	a_s	$= \delta(a_m, z_f)$

В таблице выходов на пересечении столбца a_m и строки z_f записывается выходной сигнал, который есть функция λ от a_m и z_f.

	a_m	
z_f	w_g	$= \lambda(a_m, z_f)$

Пример: Задание автомата Мили табличным способом (автомат имеет два входных сигнала, два выходных сигнала и три состояния).

Таблица переходов

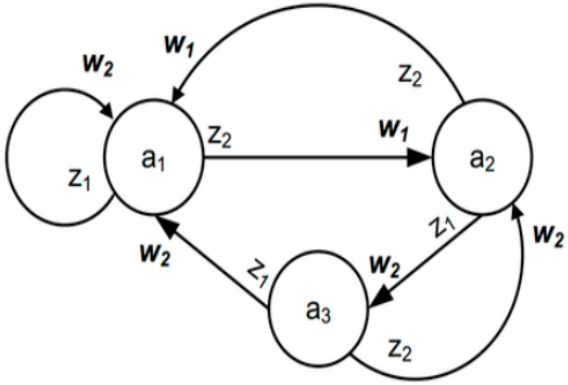
δ	a_1	a_2	a_3
z_1	a_1	a_3	a_1
z_2	a_2	a_1	a_2

Таблица выходов

λ	a_1	a_2	a_3
z_1	w_2	w_2	w_2
z_2	w_1	w_1	w_2

Графический способ задания автомата Мили

На рисунке приведен граф автомата Мили на 3 состояния, имеющий 2 входных сигнала и 2 выходных сигнала (см. предыдущий пример).



Табличный способ задания автомата Мура

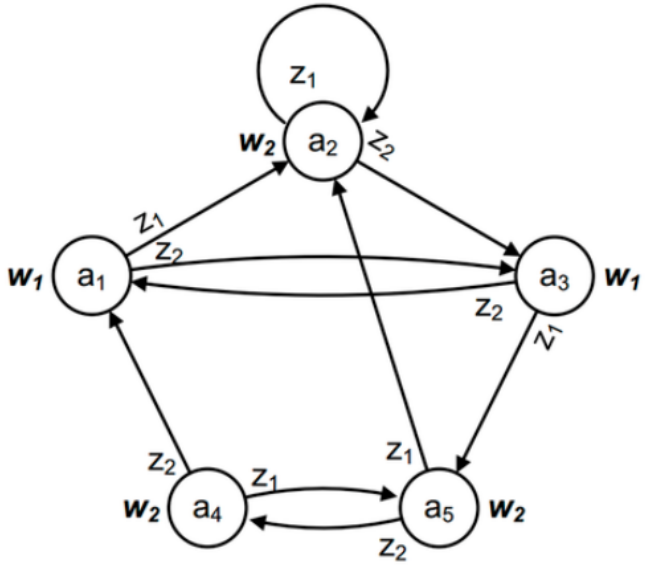
В автомате Мура выходной сигнал зависит только от состояния автомата и не зависит от входного сигнала.

Поэтому достаточно для задания автомата Мура в таблице переходов добавить одну строку.

λ	w_1	w_2	w_1	w_2	w_2
δ	a_1	a_2	a_3	a_4	a_5
z_1	a_2	a_2	a_5	a_5	a_2
z_2	a_3	a_3	a_1	a_1	a_4

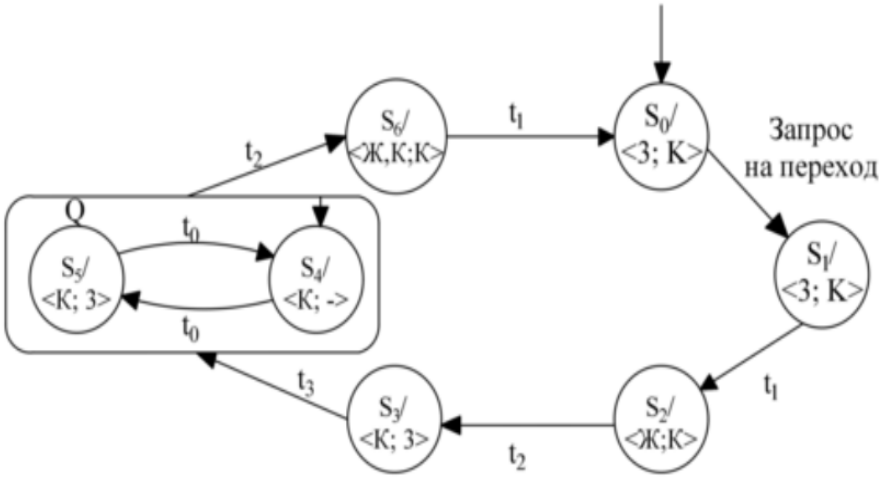
Графический способ задания автомата Мура

На рисунке приведен граф автомата Мура на 5 состояний, имеющий 2 входных сигнала и 2 выходных сигнала.



Применение автомата Мура

Автомат, регулирующий пешеходный переход



Рассмотрим автомат, регулирующий пешеходный переход по запросу пешеходов. Внешние события автомата — это события нажатия пешеходами кнопки-запроса на тротуаре и исчерпание тайм-аута. Автомат строится как автомат Мура, в котором выход — регулирование светофора и разрешающий сигнал на переход — это потенциальные сигналы, являющиеся функциями состояния.

Выход автомата в каждом состоянии определяется парой $\langle$ Светофор транспорта; светофор пешехода$\rangle$. Например, в состоянии S_1 управляющий автомат устанавливает $\langle$З; К$\rangle$, то есть включёнными зеленый свет транспорту и красный — пешеходам. В состоянии S_6 установлен $\langle$Ж, К; К$\rangle$, то есть желтый и красный свет транспорту (приготовиться) и красный — пешеходам. В начальном состоянии S_0 разрешен проезд транспорту, а пешеходам движение запрещено.

В состояниях S_4, S_5 при запрещающем сигнале транспорту зеленый сигнал пешеходам мигает каждые t_0 секунд в течение t_2 секунд. Запрос на переход принимается только в состоянии S_0, в остальных состояниях он игнорируется. Задержки (тайм-ауты $t_0 - t_3$) устанавливаются в момент перехода автомата в данное состояние, по исчерпании тайм-аута автомат переходит в следующее состояние. В гиперсостоянии Q, объединяющему пару состояний S_4 и S_5, автомат находится ровно t_2 секунд: внутренние переходы не срывают тайм-аута.

Именно для этого удобно использовать гиперсостояние Q.

Применение автомата Мили

Торговый автомат

В качестве примера применения автомата Мили рассмотрим автомат по продаже шоколадок стоимостью 20 рублей, принимающий монеты номиналом в 5 и 10 рублей и возвращающий сдачу, если это необходимо.

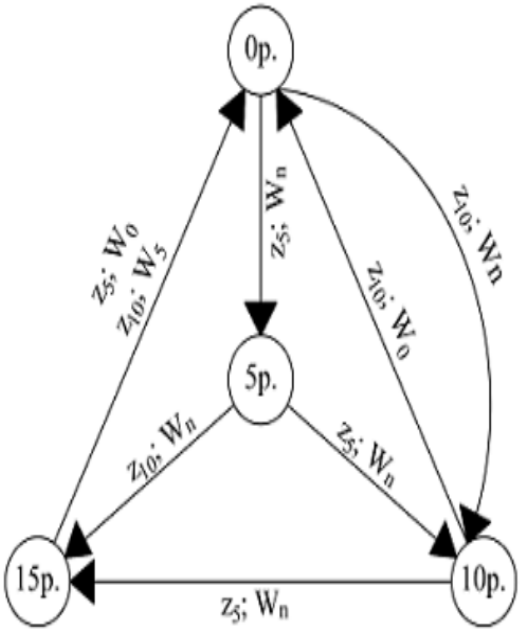
Состояний автомата всего четыре: 0, 5, 10 и 15 рублей.

Входных сигналов Z два: Z_5 — 5 рублей и Z_{10} — 10 рублей.

Выходных сигналов W три: W_n — ничего не выдавать, W_0 — выдать шоколадку и 0 рублей сдачи, W_5 — выдать шоколадку и 5 рублей сдачи.

Например, если у человека есть одна монета номиналом в 10 рублей и две монеты номиналом в 5 рублей и монеты забрасываются в порядке 10, 5 и 5 рублей, то происходит следующее:

1. Автомат переходит в состояние 10р. и ничего не выдает;
2. Автомат переходит в состояние 15р. и ничего не выдает;
3. Автомат переходит в состояние 0р., выдает шоколадку и не возвращает сдачу.



Эквивалентность автоматов Мура и Мили

Теорема (Эквивалентность автоматов Мура и Мили):

Для каждого автомата Мили может быть построен эквивалентный ему автомат Мура, и обратно — для каждого автомата Мура может быть построен эквивалентный ему автомат Мили.